

P3370R0: Add new library headers from C23

Introduction

C23 added the `<stdbit.h>` and `<stdchkint.h>` headers. This paper proposes to add these headers to C++ to increase the subset of code that compiles with C and C++.

These interfaces are necessarily C-like; this proposal does not intend to preclude adding a future C++ library facility that is in harmony with prevalent C++ style. In particular, adding `<cstdlib...>` headers is not proposed.

Type-generic macros and type-generic functions do not exist in C++, but function templates can provide the same call interface. Thus, the use of the former in C is replaced by the latter in C++.

Wording

Add `<stdbit.h>` and `<stdckdint.h>` to table [tab:c.headers] in 17.14.1 [support.c.headers.general].

Change in 17.14.7 [support.c.headers.other] as follows:

Every C header other than `<complex.h>` (17.14.2), `<iso646.h>` (17.14.3), `<stdalign.h>` (17.14.4), `<stdatomic.h>` (33.5.12), `<stdbit.h>` ([`stdbit.h.syn`]), `<stdbool.h>` (17.14.5), `<stdckdint.h>` ([`stdchkint.h.syn`]), and `<tgmath.h>` (17.14.6), each of which has a name of the form `<name.h>`, behaves as if each name placed in the standard library namespace by the corresponding `<cname>` header is placed within the global namespace scope, except for the functions described in 28.7.6, the `std::lerp` function overloads (28.7.4), the declaration of `std::byte` (17.2.1), and the functions and function templates described in 17.2.5. It is unspecified whether these names are first declared or defined within namespace scope (6.4.6) of the namespace `std` and are then injected into the global namespace scope by explicit *using-declarations* (9.9).

Add a new subclause to clause 22 [utilities] as follows:

22.17 Header `<stdbit.h>` synopsis [`stdbit.h.syn`]

```
#define __STDC_VERSION_STDBIT_H__ 202311L

#define __STDC_ENDIAN_BIG__          /* see below */
#define __STDC_ENDIAN_LITTLE__      /* see below */
#define __STDC_ENDIAN_NATIVE__      /* see below */

unsigned int stdc_leading_zeros_uc(unsigned char value);
unsigned int stdc_leading_zeros_us(unsigned short value);
unsigned int stdc_leading_zeros_ui(unsigned int value);
unsigned int stdc_leading_zeros_ul(unsigned long int value);
unsigned int stdc_leading_zeros_ull(unsigned long long int value);
template<class T> /* see below */ stdc_leading_zeros(T value);

unsigned int stdc_leading_ones_uc(unsigned char value);
unsigned int stdc_leading_ones_us(unsigned short value);
unsigned int stdc_leading_ones_ui(unsigned int value);
```

```

unsigned int stdc_leading_ones_ul(unsigned long int value);
unsigned int stdc_leading_ones_ull(unsigned long long int value);
template<class T> /* see below */ stdc_leading_ones(T value);

unsigned int stdc_trailing_zeros_uc(unsigned char value);
unsigned int stdc_trailing_zeros_us(unsigned short value);
unsigned int stdc_trailing_zeros_ui(unsigned int value);
unsigned int stdc_trailing_zeros_ul(unsigned long int value);
unsigned int stdc_trailing_zeros_ull(unsigned long long int value);
template<class T> /* see below */ stdc_trailing_zeros(T value);

unsigned int stdc_trailing_ones_uc(unsigned char value);
unsigned int stdc_trailing_ones_us(unsigned short value);
unsigned int stdc_trailing_ones_ui(unsigned int value);
unsigned int stdc_trailing_ones_ul(unsigned long int value);
unsigned int stdc_trailing_ones_ull(unsigned long long int value);
template<class T> /* see below */ stdc_trailing_ones(T value);

unsigned int stdc_first_leading_zero_uc(unsigned char value);
unsigned int stdc_first_leading_zero_us(unsigned short value);
unsigned int stdc_first_leading_zero_ui(unsigned int value);
unsigned int stdc_first_leading_zero_ul(unsigned long int value);
unsigned int stdc_first_leading_zero_ull(unsigned long long int value);
template<class T> /* see below */ stdc_first_leading_zero(T value);

unsigned int stdc_first_leading_one_uc(unsigned char value);
unsigned int stdc_first_leading_one_us(unsigned short value);
unsigned int stdc_first_leading_one_ui(unsigned int value);
unsigned int stdc_first_leading_one_ul(unsigned long int value);
unsigned int stdc_first_leading_one_ull(unsigned long long int value);
template<class T> /* see below */ stdc_first_leading_one(T value);

unsigned int stdc_first_trailing_zero_uc(unsigned char value);
unsigned int stdc_first_trailing_zero_us(unsigned short value);
unsigned int stdc_first_trailing_zero_ui(unsigned int value);
unsigned int stdc_first_trailing_zero_ul(unsigned long int value);
unsigned int stdc_first_trailing_zero_ull(unsigned long long int value);
template<class T> /* see below */ stdc_first_trailing_zero(T value);

unsigned int stdc_first_trailing_one_uc(unsigned char value);
unsigned int stdc_first_trailing_one_us(unsigned short value);
unsigned int stdc_first_trailing_one_ui(unsigned int value);
unsigned int stdc_first_trailing_one_ul(unsigned long int value);
unsigned int stdc_first_trailing_one_ull(unsigned long long int value);
template<class T> /* see below */ stdc_first_trailing_one(T value);

unsigned int stdc_count_zeros_uc(unsigned char value);
unsigned int stdc_count_zeros_us(unsigned short value);
unsigned int stdc_count_zeros_ui(unsigned int value);
unsigned int stdc_count_zeros_ul(unsigned long int value);
unsigned int stdc_count_zeros_ull(unsigned long long int value);
template<class T> /* see below */ stdc_count_zeros(T value);

unsigned int stdc_count_ones_uc(unsigned char value);
unsigned int stdc_count_ones_us(unsigned short value);
unsigned int stdc_count_ones_ui(unsigned int value);
unsigned int stdc_count_ones_ul(unsigned long int value);
unsigned int stdc_count_ones_ull(unsigned long long int value);
template<class T> /* see below */ stdc_count_ones(T value);

bool stdc_has_single_bit_uc(unsigned char value);
bool stdc_has_single_bit_us(unsigned short value);
bool stdc_has_single_bit_ui(unsigned int value);
bool stdc_has_single_bit_ul(unsigned long int value);
bool stdc_has_single_bit_ull(unsigned long long int value);

```

```

template<class T> bool stdc_has_single_bit(T value);

unsigned int stdc_bit_width_uc(unsigned char value);
unsigned int stdc_bit_width_us(unsigned short value);
unsigned int stdc_bit_width_ui(unsigned int value);
unsigned int stdc_bit_width_ul(unsigned long int value);
unsigned int stdc_bit_width_ull(unsigned long long int value);
template<class T> /* see below */ stdc_bit_width(T value);

unsigned char stdc_bit_floor_uc(unsigned char value);
unsigned short stdc_bit_floor_us(unsigned short value);
unsigned int stdc_bit_floor_ui(unsigned int value);
unsigned long int stdc_bit_floor_ul(unsigned long int value);
unsigned long long int stdc_bit_floor_ull(unsigned long long int value);
template<class T> T stdc_bit_floor(T value);

unsigned char stdc_bit_ceil_uc(unsigned char value);
unsigned short stdc_bit_ceil_us(unsigned short value);
unsigned int stdc_bit_ceil_ui(unsigned int value);
unsigned long int stdc_bit_ceil_ul(unsigned long int value);
unsigned long long int stdc_bit_ceil_ull(unsigned long long int value);
template<class T> T stdc_bit_ceil(T value);

```

For a function template whose return type is not specified above, the return type is an implementation-defined unsigned integer type large enough to represent all possible result values. Each function template has the same semantics as the corresponding type-generic function with the same name specified in ISO/IEC 9899:2024, 7.18.

Mandates: T is an unsigned integer type.

Otherwise, the contents and meaning of the header <stdbit.h> are the same as the C standard library header <stdbit.h>.

See also: ISO/IEC 9899:2024, 7.18

Add a new subclause to clause 28 [numerics] as follows:

28.10 C compatibility [numerics.c]

28.10.1 Header <stdckdint.h> synopsis [stdckdint.h.syn]

```

#define __STDC_VERSION_STDCKDINT_H__ 202311L

template<class type1, class type2, class type3>
bool ckd_add(type1 *result, type2 a, type3 b);
template<class type1, class type2, class type3>
bool ckd_sub(type1 *result, type2 a, type3 b);
template<class type1, class type2, class type3>
bool ckd_mul(type1 *result, type2 a, type3 b);

```

The contents and meaning of the header <stdchkint.h> are the same as the C standard library header <stdchkint.h>, except as specified in [numerics.c.ckdint].

See also: ISO/IEC 9899:2024, 7.20

28.10.2 Checked integer operations [numerics.c.ckdint]

```

template<class type1, class type2, class type3>
bool ckd_add(type1 *result, type2 a, type3 b);

```

```
template<class type1, class type2, class type3>  
bool ckd_sub(type1 *result, type2 a, type3 b);  
template<class type1, class type2, class type3>  
bool ckd_mul(type1 *result, type2 a, type3 b);
```

Mandates: Each of the types type1, type2, and type3 is a cv-unqualified signed or unsigned integer type.

Remarks: Each function template has the same semantics as the corresponding type-generic macro with the same name specified in ISO/IEC 9899:2024, 7.20.